

1) What operating system (including revision) did you use for your 8051 code development?

Windows 24H2

2) What compiler (including revision) did you use? Submit the command line options you used when building code.

SDCC : mcs51 2.3.1-pj3 (Jan 20 2002) (MINGW)

3) What exactly (include name/revision if appropriate) did you use to build your code (what IDE, make/makefile, or command line)? Submit any makefiles or batch files used.

For this lab I have used make with batchisp integrated into the make file

```
# -----
```

```
# Project: Lab_3_part_2
```

```
# Author: Jack Haggard
```

```
# Toolchain: SDCC 4.2.0
```

```
# Make: GNU Make 3.77
```

```
#
```

```
# NOTE:
```

```
# If SDCC is not found, update SDCC_PATH variable below.
```

```
# -----
```

```
SDCC_PATH = "C:\Program Files\SDCC\bin\sdcc.exe"
```

```
CC = $(SDCC_PATH)
```

```
batchisp_PATH = "C:\Program Files (x86)\Atmel\Flip 3.4.7\bin\batchisp.exe"
```

```
PORT = "COM6"
```

```
paulmon_path = "C:\College\ECEN5613\as31-sdcc231-pj3-win32\sdcc\bin"
```

```
BAUD = "115200"
```

putty:

```
putty.exe -serial $(PORT) $(BAUD)
```

flash:

```
$(batchisp_PATH) -device at89c51rc2 -hardware RS232 -port $(PORT) -baudrate
```

```
$(BAUD) -operation erase f memory flash blankcheck loadbuffer
```

```
$(paulmon_path)/paulmon21.hex program verify
```

```
$(batchisp_PATH) -device at89c51rc2 -hardware RS232 -port $(PORT) -baudrate
```

```
$(BAUD) -operation memory flash loadbuffer $(paulmon_path)/extra.hex program verify
```

```
$(batchisp_PATH) -device at89c51rc2 -hardware RS232 -port $(PORT) -baudrate
```

```
$(BAUD) -operation memory flash loadbuffer bin/output.hex program verify
```

```
putty.exe -serial $(PORT) $(BAUD)
```

paulmon: lib/paulmon21.asm

paulmon_extra: lib/extra.asm

all: bin/output.hex

```
# Link all .rel object files into final executable (.hex)
# --code-loc : The start location of the user code segment
# --code-size : Causes the linker to check if the code memory usage is within limits of the given
value
# --xram-loc : The start location of the external ram (this is set to 0x0001 to avoid malloc
failures if heap starts at 0x0000 )
# --xram-size : Total external RAM size
# --model-large : Generate code for Large model programs
# --out-fmt-ihx : Generate Intel HEX output format
```

```
bin/output.hex: bin/main.rel bin/my_serial.rel bin/i2c.rel
$(CC) bin/main.rel bin/my_serial.rel bin/i2c.rel \
--code-loc 0x2000 --code-size 0x6000 \
--xram-loc 0x0001 --xram-size 0x7FFF \
--model-large --out-fmt-ihx -o bin/output.hex
```

```
# Compile main application file
# -c : --compile-only will compile and assemble the source, but will not call the linkage editor
# -mmcs51 : Generate code for the Intel MCS51 family of processors. This is the default
processor target
# --model-large : Generate code for Large model programs
# -I headers : Add headers directory to include path
```

```
bin/main.rel: src/main.c headers/my_serial.h headers/main.h headers/i2c.h
$(CC) -c -mmcs51 --model-large -I headers src/main.c -o bin/main.rel
```

```
bin/i2c.rel: src/i2c.c headers/i2c.h
$(CC) -c -mmcs51 --model-large -I headers src/i2c.c -o bin/i2c.rel
```

```
bin/my_serial.rel: src/my_serial.c headers/my_serial.h
$(CC) -c -mmcs51 --model-large -I headers src/my_serial.c -o bin/my_serial.rel
```

```
# Delete all generated files from bin directory
clean:
    del /q bin\*.*
```

4) Did you install and use any other software tools to complete your lab assignment?
SDCC, AS31, Paulmon

5) Did you experience any problems or challenges with this lab assignment or any of the software tools? If so, describe the issues.

I2C was particularly difficult to implement despite having access to publicly available code libraries.

6) If you have any suggestions for changes to this lab assignment for the future, please include those ideas in your submission.

I don't have any suggestions at this time.